

# Sampling

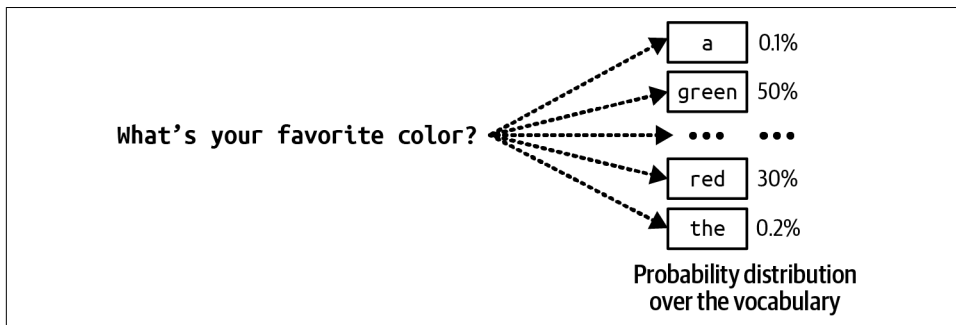
A model constructs its outputs through a process known as *sampling*. This section discusses different sampling strategies and *sampling variables*, including temperature, top-k, and top-p. It'll then explore how to sample multiple outputs to improve a model's performance. We'll also see how the sampling process can be modified to get models to generate responses that follow certain formats and constraints.

Sampling makes AI's outputs probabilistic. Understanding this probabilistic nature is important for handling AI's behaviors, such as inconsistency and hallucination. This section ends with a deep dive into what this probabilistic nature means and how to work with it.

## Sampling Fundamentals

Given an input, a neural network produces an output by first computing the probabilities of possible outcomes. For a classification model, possible outcomes are the available classes. As an example, if a model is trained to classify whether an email is spam or not, there are only two possible outcomes: spam and not spam. The model computes the probability of each of these two outcomes—e.g., the probability of the email being spam is 90%, and not spam is 10%. You can then make decisions based on these output probabilities. For example, if you decide that any email with a spam probability higher than 50% should be marked as spam, an email with a 90% spam probability will be marked as spam.

For a language model, to generate the next token, the model first computes the probability distribution over all tokens in the vocabulary, which looks like [Figure 2-14](#).



*Figure 2-14. To generate the next token, the language model first computes the probability distribution over all tokens in the vocabulary.*

When working with possible outcomes of different probabilities, a common strategy is to pick the outcome with the highest probability. Always picking the most likely outcome = is called *greedy sampling*. This often works for classification tasks. For example, if the model thinks that an email is more likely to be spam than not spam, it makes sense to mark it as spam. However, for a language model, greedy sampling creates boring outputs. Imagine a model that, for whatever question you ask, always responds with the most common words.

Instead of always picking the next most likely token, the model can sample the next token according to the probability distribution over all possible values. Given the context of “My favorite color is ...” as shown in [Figure 2-14](#), if “red” has a 30% chance of being the next token and “green” has a 50% chance, “red” will be picked 30% of the time, and “green” 50% of the time.

How does a model compute these probabilities? Given an input, a neural network outputs a logit vector. Each *logit* corresponds to one possible value. In the case of a language model, each logit corresponds to one token in the model’s vocabulary. The logit vector size is the size of the vocabulary. A visualization of the logits vector is shown in [Figure 2-15](#).

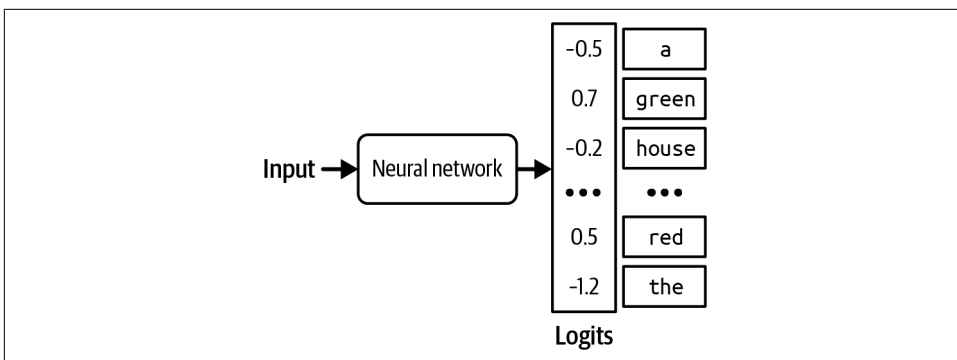


Figure 2-15. For each input, a language model produces a logit vector. Each logit corresponds to a token in the vocabulary.

While larger logits correspond to higher probabilities, logits don't represent probabilities. Logits don't sum up to one. Logits can even be negative, while probabilities have to be non-negative. To convert logits to probabilities, a softmax layer is often used. Let's say the model has a vocabulary of  $N$  and the logit vector is  $[x_1, x_2, \dots, x_N]$ . The probability for the  $i^{\text{th}}$  token,  $p_i$  is computed as follows:

$$p_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

## Sampling Strategies

The right sampling strategy can make a model generate responses more suitable for your application. For example, one sampling strategy can make the model generate more creative responses, whereas another strategy can make its generations more predictable. Many different sample strategies have been introduced to nudge models toward responses with specific attributes. You can also design your own sampling strategy, though this typically requires access to the model's logits. Let's go over a few common sampling strategies to see how they work.

### Temperature

One problem with sampling the next token according to the probability distribution is that the model can be less creative. In the previous example, common colors like "red", "green", "purple", and so on have the highest probabilities. The language model's answer ends up sounding like that of a five-year-old: "My favorite color is green". Because "the" has a low probability, the model has a low chance of generating a creative sentence such as "My favorite color is the color of a still lake on a spring morning".

To redistribute the probabilities of the possible values, you can sample with a *temperature*. Intuitively, a higher temperature reduces the probabilities of common tokens, and as a result, increases the probabilities of rarer tokens. This enables models to create more creative responses.

Temperature is a constant used to adjust the logits before the softmax transformation. Logits are divided by temperature. For a given temperature  $T$ , the adjusted logit for the  $i^{\text{th}}$  token is  $\frac{x_i}{T}$ . Softmax is then applied on this adjusted logit instead of on  $x_i$ .

Let's walk through a simple example to examine the effect of temperature on probabilities. Imagine that we have a model that has only two possible outputs: A and B. The logits computed from the last layer are [1, 2]. The logit for A is 1 and B is 2.

Without using temperature, which is equivalent to using the temperature of 1, the softmax probabilities are [0.27, 0.73]. The model picks B 73% of the time.

With temperature = 0.5, the probabilities are [0.12, 0.88]. The model now picks B 88% of the time.

The higher the temperature, the less likely it is that the model is going to pick the most obvious value (the value with the highest logit), making the model's outputs more creative but potentially less coherent. The lower the temperature, the more likely it is that the model is going to pick the most obvious value, making the model's output more consistent but potentially more boring.<sup>24</sup>

Figure 2-16 shows the softmax probabilities for tokens A and B at different temperatures. As the temperature gets closer to 0, the probability that the model picks token B becomes closer to 1. In our example, for a temperature below 0.1, the model almost always outputs B. As the temperature increases, the probability that token A is picked increases while the probability that token B is picked decreases. Model providers typically limit the temperature to be between 0 and 2. If you own your model, you can use any non-negative temperature. A temperature of 0.7 is often recommended for creative use cases, as it balances creativity and predictability, but you should experiment and find the temperature that works best for you.

---

<sup>24</sup> A visual image I have in mind when thinking about temperature, which isn't entirely scientific, is that a higher temperature causes the probability distribution to be more chaotic, which enables lower-probability tokens to surface.

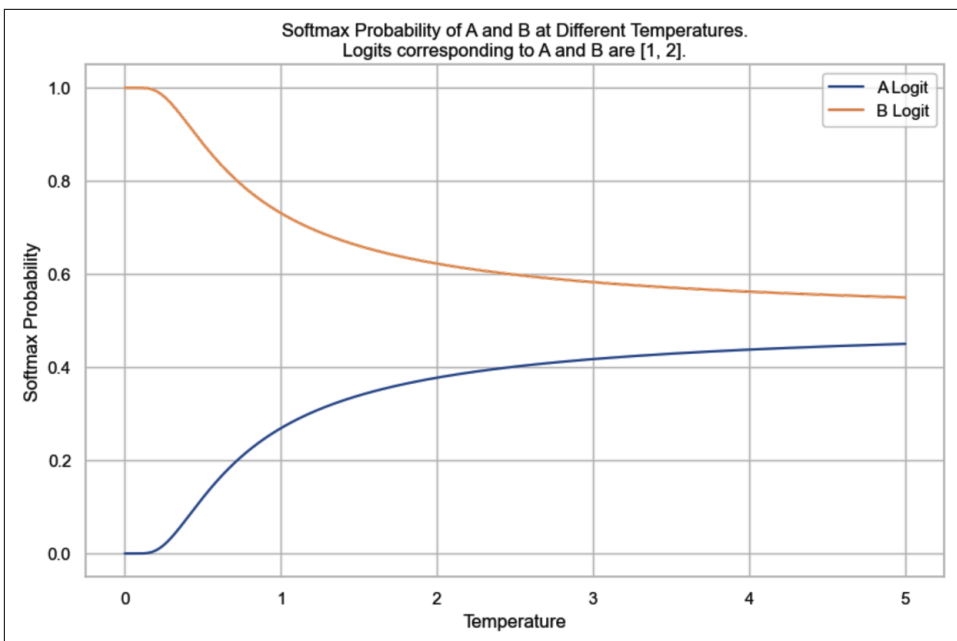


Figure 2-16. The softmax probabilities for tokens A and B at different temperatures, given their logits being [1, 2]. Without setting the temperature value, which is equivalent to using the temperature of 1, the softmax probability of B would be 73%.

It's common practice to set the temperature to 0 for the model's outputs to be more consistent. Technically, temperature can never be 0—logits can't be divided by 0. In practice, when we set the temperature to 0, the model just picks the token with the largest logit,<sup>25</sup> without doing logit adjustment and softmax calculation.

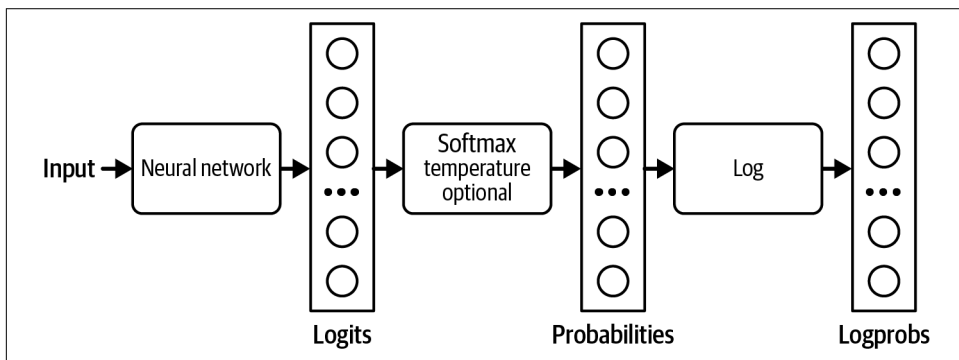


A common debugging technique when working with an AI model is to look at the probabilities this model computes for given inputs. For example, if the probabilities look random, the model hasn't learned much.

<sup>25</sup> Performing an **arg max** function.

Many model providers return probabilities generated by their models as **logprobs**. *Logprobs*, short for *log probabilities*, are probabilities in the log scale. Log scale is preferred when working with a neural network's probabilities because it helps reduce the **underflow** problem.<sup>26</sup> A language model might be working with a vocabulary size of 100,000, which means the probabilities for many of the tokens can be too small to be represented by a machine. The small numbers might be rounded down to 0. Log scale helps reduce this problem.

**Figure 2-17** shows the workflow of how logits, probabilities, and logprobs are computed.



*Figure 2-17. How logits, probabilities, and logprobs are computed.*

As you'll see throughout the book, logprobs are useful for building applications (especially for classification), evaluating applications, and understanding how models work under the hood. However, as of this writing, many model providers don't expose their models' logprobs, or if they do, the logprobs API is limited.<sup>27</sup> The limited logprobs API is likely due to security reasons as a model's exposed logprobs make it easier for others to replicate the model.

---

<sup>26</sup> The underflow problem occurs when a number is too small to be represented in a given format, leading to it being rounded down to zero.

<sup>27</sup> To be more specific, as of this writing, OpenAI API only shows you the **logprobs** of up to the 20 most likely tokens. It used to let you get the logprobs of arbitrary user-provided text but discontinued this in **September 2023**. Anthropic doesn't expose its models' logprobs.

## Top-k

*Top-k* is a sampling strategy to reduce the computation workload without sacrificing too much of the model's response diversity. Recall that a softmax layer is used to compute the probability distribution over all possible values. Softmax requires two passes over all possible values: one to perform the exponential sum  $\sum_j e^{x_j}$ , and one to perform  $\frac{e^{x_i}}{\sum_j e^{x_j}}$  for each value. For a language model with a large vocabulary, this process is computationally expensive.

To avoid this problem, after the model has computed the logits, we pick the top-k logits and perform softmax over these top-k logits only. Depending on how diverse you want your application to be, k can be anywhere from 50 to 500—much smaller than a model's vocabulary size. The model then samples from these top values. A smaller k value makes the text more predictable but less interesting, as the model is limited to a smaller set of likely words.

## Top-p

In top-k sampling, the number of values considered is fixed to k. However, this number should change depending on the situation. For example, given the prompt “Do you like music? Answer with only yes or no.” the number of values considered should be two: yes and no. Given the prompt “What’s the meaning of life?” the number of values considered should be much larger.

*Top-p*, also known as *nucleus sampling*, allows for a more dynamic selection of values to be sampled from. In top-p sampling, the model sums the probabilities of the most likely next values in descending order and stops when the sum reaches p. Only the values within this cumulative probability are considered. Common values for top-p (nucleus) sampling in language models typically range from 0.9 to 0.95. A top-p value of 0.9, for example, means that the model will consider the smallest set of values whose cumulative probability exceeds 90%.

Let's say the probabilities of all tokens are as shown in [Figure 2-18](#). If top-p is 90%, only “yes” and “maybe” will be considered, as their cumulative probability is greater than 90%. If top-p is 99%, then “yes”, “maybe”, and “no” are considered.

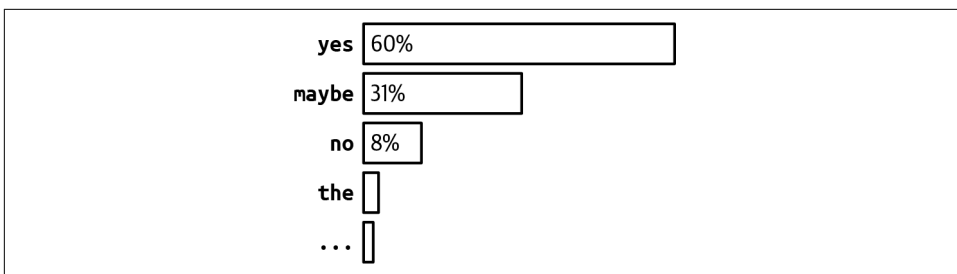


Figure 2-18. Example token probabilities.

Unlike top-k, top-p doesn't necessarily reduce the softmax computation load. Its benefit is that because it focuses only on the set of most relevant values for each context, it allows outputs to be more contextually appropriate. In theory, there don't seem to be a lot of benefits to top-p sampling. However, in practice, top-p sampling has proven to work well, causing its popularity to rise.

A related sampling strategy is **min-p**, where you set the minimum probability that a token must reach to be considered during sampling.

### Stopping condition

An autoregressive language model generates sequences of tokens by generating one token after another. A long output sequence takes more time, costs more compute (money),<sup>28</sup> and can sometimes annoy users. We might want to set a condition for the model to stop the sequence.

One easy method is to ask models to stop generating after a fixed number of tokens. The downside is that the output is likely to be cut off mid-sentence. Another method is to use *stop tokens* or *stop words*. For example, you can ask a model to stop generating when it encounters the end-of-sequence token. Stopping conditions are helpful to keep latency and costs down.

The downside of early stopping is that if you want models to generate outputs in a certain format, premature stopping can cause outputs to be malformed. For example, if you ask the model to generate JSON, early stopping can cause the output JSON to be missing things like closing brackets, making the generated JSON hard to parse.

---

<sup>28</sup> Paid model APIs often charge per number of output tokens.

## Test Time Compute

The last section discussed how a model might sample the next token. This section discusses how a model might sample the whole output.

One simple way to improve a model's response quality is *test time compute*: instead of generating only one response per query, you generate multiple responses to increase the chance of good responses. One way to do test time compute is the best of N technique discussed earlier in this chapter—you randomly generate multiple outputs and pick one that works best. However, you can also be more strategic about how to generate multiple outputs. For example, instead of generating all outputs independently, which might include many less promising candidates, you can use **beam search** to generate a fixed number of most promising candidates (the beam) at each step of sequence generation.

A simple strategy to increase the effectiveness of test time compute is to increase the diversity of the outputs, because a more diverse set of options is more likely to yield better candidates. If you use the same model to generate different options, it's often a good practice to vary the model's sampling variables to diversify its outputs.

Although you can usually expect some model performance improvement by sampling multiple outputs, it's expensive. On average, generating two outputs costs approximately twice as much as generating one.<sup>29</sup>



I use the term *test time compute* to be consistent with the existing literature, even though several early reviewers protested that this term is confusing. In AI research, test time is typically used to refer to inference because researchers mostly only do inference to test a model. However, this technique can be applied to models in production in general. It's test time compute because the number of outputs you can sample is determined by how much compute you can allocate to each inference call.

To pick the best output, you can either show users multiple outputs and let them choose the one that works best for them, or you can devise a method to select the best one. One selection method is to pick the output with the highest probability. A language model's output is a sequence of tokens, and each token has a probability computed by the model. The probability of an output is the product of the probabilities of all tokens in the output.

---

<sup>29</sup> There are things you can do to reduce the cost of generating multiple outputs for the same input. For example, the input might only be processed once and reused for all outputs.

Consider the sequence of tokens [“I”, “love”, “food”]. If the probability for “I” is 0.2, the probability for “love” given “I” is 0.1, and the probability for “food” given “I” and “love” is 0.3, the sequence’s probability is:  $0.2 \times 0.1 \times 0.3 = 0.006$ . Mathematically, this can be denoted as follows:

$$p(I \text{ love food}) = p(I) \times p(I \mid \text{love}) \times p(\text{food} \mid I, \text{love})$$

Remember that it’s easier to work with probabilities on a log scale. The logarithm of a product is equal to a sum of logarithms, so the logprob of a sequence of tokens is the sum of the logprob of all tokens in the sequence:

$$\text{logprob}(I \text{ love food}) = \text{logprob}(I) + \text{logprob}(I \mid \text{love}) + \text{logprob}(\text{food} \mid I, \text{love})$$

With summing, longer sequences are likely to have a lower total logprob (logprob values are usually negative, because log of values between 0 and 1 is negative). To avoid biasing toward short sequences, you can use the average logprob by dividing the sum of a sequence by its length. After sampling multiple outputs, you pick the one with the highest average logprob. As of this writing, this is what the OpenAI API uses.<sup>30</sup>

Another selection method is to use a reward model to score each output, as discussed in the previous section. Recall that both **Stitch Fix** and **Grab** pick the outputs given high scores by their reward models or verifiers. **Nextdoor** found that using a reward model was the key factor in improving their application’s performance (2023).

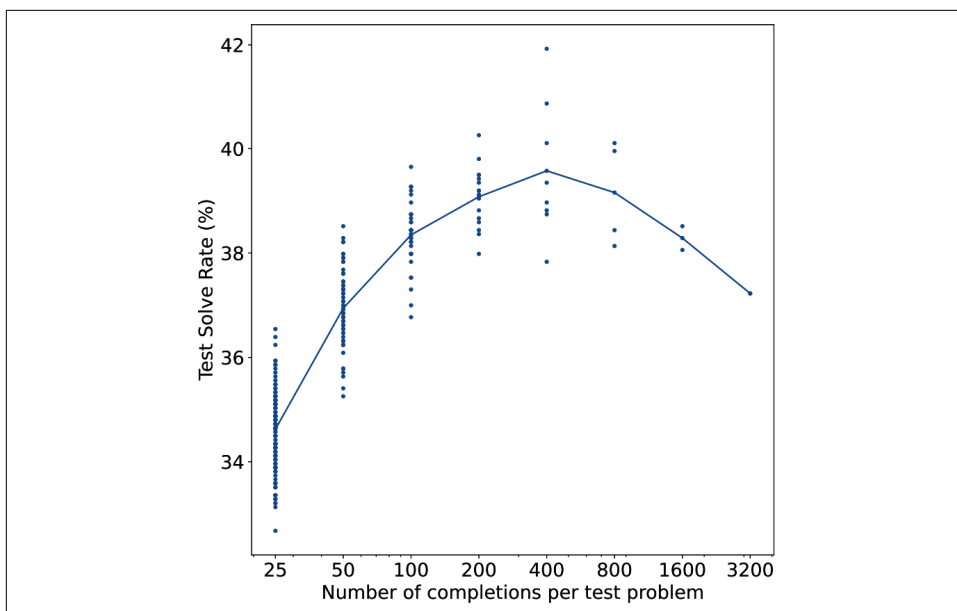
OpenAI also trained verifiers to help their models pick the best solutions to math problems (Cobbe et al., 2021). They found that using a verifier significantly boosted the model performance. *In fact, the use of verifiers resulted in approximately the same performance boost as a 30× model size increase.* This means that a 100-million-parameter model that uses a verifier can perform on par with a 3-billion-parameter model that doesn’t use a verifier.

DeepMind further proves the value of test time compute, arguing that scaling test time compute (e.g., allocating more compute to generate more outputs during inference) can be more efficient than scaling model parameters (Snell et al., 2024). The same paper asks an interesting question: If an LLM is allowed to use a fixed but non-trivial amount of inference-time compute, how much can it improve its performance on a challenging prompt?

---

30 As of this writing, in the OpenAI API, you can set the parameter **best\_of** to a specific value, say 10, to ask OpenAI models to return the output with the highest average logprob out of 10 different outputs.

In OpenAI’s experiment, sampling more outputs led to better performance, but only up to a certain point. In this experiment, that point was 400 outputs. Beyond this point, performance decreases, as shown in [Figure 2-19](#). They hypothesized that as the number of sampled outputs increases, the chance of finding adversarial outputs that can fool the verifier also increases. However, a Stanford experiment showed a different conclusion. “Monkey Business” ([Brown et al., 2024](#)) finds that the number of problems solved often increases log-linearly as the number of samples increases from 1 to 10,000. While it’s interesting to think about whether test time compute can be scaled indefinitely, I don’t believe anyone in production samples 400 or 10,000 different outputs for each input. The cost would be astronomical.



*Figure 2-19. [OpenAI \(2021\)](#) found that sampling more outputs led to better performance, but only up to 400 outputs.*

You can also use application-specific heuristics to select the best response. For example, if your application benefits from shorter responses, you can pick the shortest candidate. If your application converts natural language to SQL queries, you can get the model to keep on generating outputs until it generates a valid SQL query.

One particularly interesting application of test time compute is to overcome the latency challenge. For some queries, especially chain-of-thought queries, a model might take a long time to complete the response. Kittipat Kampa, head of AI at TIFIN, told me that his team asks their model to generate multiple responses in parallel and show the user the first response that is completed and valid.

Picking out the most common output among a set of outputs can be especially useful for tasks that expect exact answers.<sup>31</sup> For example, given a math problem, the model can solve it multiple times and pick the most frequent answer as its final solution. Similarly, for a multiple-choice question, a model can pick the most frequent output option. This is what Google did when evaluating Gemini on the MMLU benchmark. They sampled 32 outputs for each question. This allowed the model to achieve a higher score than what it would've achieved with only one output per question.

A model is considered robust if it doesn't dramatically change its outputs with small variations in the input. The less robust a model is, the more you can benefit from sampling multiple outputs.<sup>32</sup> For one project, we used AI to extract certain information from an image of the product. We found that for the same image, our model could read the information only half of the time. For the other half, the model said that the image was too blurry or the text was too small to read. However, by trying three times with each image, the model was able to extract the correct information for most images.

## Structured Outputs

Often, in production, you need models to generate outputs following certain formats. Structured outputs are crucial for the following two scenarios:

1. *Tasks requiring structured outputs.* The most common category of tasks in this scenario is semantic parsing. Semantic parsing involves converting natural language into a structured, machine-readable format. Text-to-SQL is an example of semantic parsing, where the outputs must be valid SQL queries. Semantic parsing allow users to interact with APIs using a natural language (e.g., English). For example, text-to-PostgreSQL allows users to query a Postgres database using English queries such as “What’s the average monthly revenue over the last 6 months” instead of writing it in PostgreSQL.

---

<sup>31</sup> Wang et al. (2023) called this approach self-consistency.

<sup>32</sup> The optimal thing to do with a brittle model, however, is to swap it out for another.

This is an example of a prompt for GPT-4o to do text-to-regex. The outputs are actual outputs generated by GPT-4o:

**System prompt**

Given an item, create a regex that represents all the ways the item can be written. Return only the regex.

Example:

US phone number -> \+?1?\s?(\(\)?(\d{3})(\d{1}\))[-.\s]?(\d{3})[-.\s]?(\d{4})

**User prompt**

Email address ->

**GPT-4o**

[a-zA-Z0-9.\_%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

**User prompt**

Dates ->

**GTP-4o**

(?:\d{1,2}[/\.-])(?:\d{1,2}[/\.-])?\d{2,4}

Other categories of tasks in this scenario include classification where the outputs have to be valid classes.

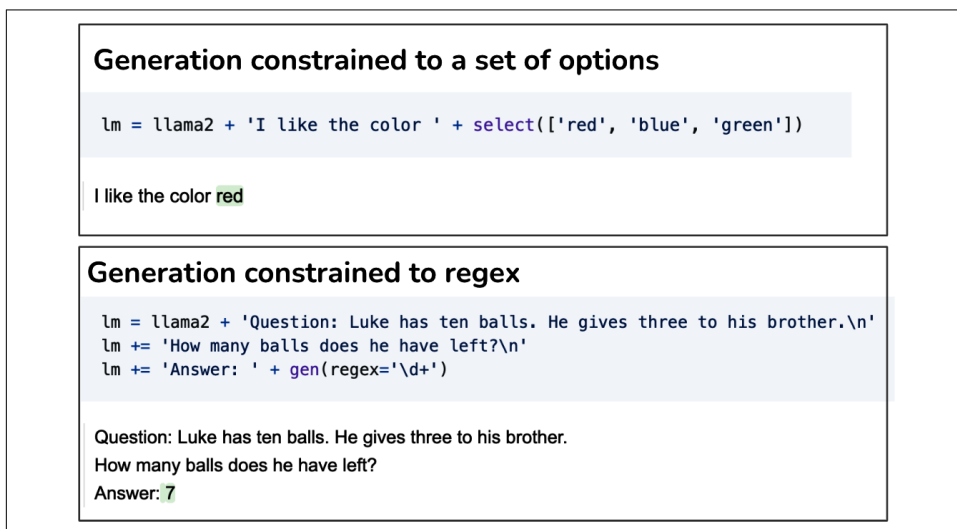
2. *Tasks whose outputs are used by downstream applications.* In this scenario, the task itself doesn't need the outputs to be structured, but because the outputs are used by other applications, they need to be parsable by these applications.

For example, if you use an AI model to write an email, the email itself doesn't have to be structured. However, a downstream application using this email might need it to be in a specific format—for example, a JSON document with specific keys, such as {"title": [TITLE], "body": [EMAIL BODY]}.

*This is especially important for agentic workflows* where a model's outputs are often passed as inputs into tools that the model can use, as discussed in [Chapter 6](#).

Frameworks that support structured outputs include **guidance**, **outlines**, **instructor**, and **llama.cpp**. Each model provider might also use their own techniques to improve their models' ability to generate structured outputs. OpenAI was the first model provider to introduce **JSON mode** in their text generation API. Note that an API's JSON mode typically guarantees only that the outputs are valid JSON—not the content of the JSON objects. The otherwise valid generated JSONs can also be truncated, and thus not parsable, if the generation stops too soon, such as when it reaches the maximum output token length. However, if the max token length is set too long, the model's responses become both too slow and expensive.

**Figure 2-20** shows two examples of using guidance to generate outputs constrained to a set of options and a regex.



*Figure 2-20. Using guidance to generate constrained outputs.*

You can guide a model to generate structured outputs at different layers of the AI stack: prompting, post-processing, test time compute, constrained sampling, and finetuning. The first three are more like bandages. They work best if the model is already pretty good at generating structured outputs and just needs a little nudge. For intensive treatment, you need constrained sampling and finetuning.

Test time compute has just been discussed in the previous section—keep on generating outputs until one fits the expected format. This section focuses on the other four approaches.

## Prompting

Prompting is the first line of action for structured outputs. You can instruct a model to generate outputs in any format. However, whether a model can follow this instruction depends on the model's instruction-following capability (discussed in [Chapter 4](#)), and the clarity of the instruction (discussed in [Chapter 5](#)). While models are getting increasingly good at following instructions, there's no guarantee that they'll always follow your instructions.<sup>33</sup> A few percentage points of invalid model outputs can still be unacceptable for many applications.

To increase the percentage of valid outputs, some people use AI to validate and/or correct the output of the original prompt. This is an example of the AI as a judge approach discussed in [Chapter 3](#). This means that for each output, there will be at least two model queries: one to generate the output and one to validate it. While the added validation layer can significantly improve the validity of the outputs, the extra cost and latency incurred by the extra validation queries can make this approach too expensive for some.

## Post-processing

Post-processing is simple and cheap but can work surprisingly well. During my time teaching, I noticed that students tended to make very similar mistakes. When I started working with foundation models, I noticed the same thing. A model tends to repeat similar mistakes across queries. This means if you find the common mistakes a model makes, you can potentially write a script to correct them. For example, if the generated JSON object misses a closing bracket, manually add that bracket. LinkedIn's defensive YAML parser increased the percentage of correct YAML outputs from 90% to 99.99% ([Bottaro and Ramgopal, 2020](#)).



JSON and YAML are common text formats. LinkedIn found that their underlying model, GPT-4, worked with both, but they chose YAML as their output format because it is less verbose, and hence requires fewer output tokens than JSON ([Bottaro and Ramgopal, 2020](#)).

Post-processing works only if the mistakes are easy to fix. This usually happens if a model's outputs are already mostly correctly formatted, with occasional small errors.

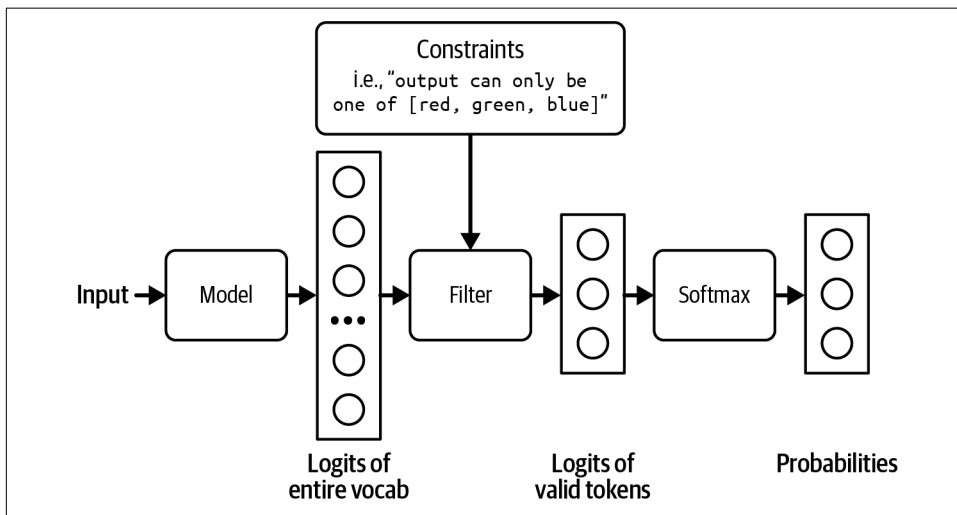
---

<sup>33</sup> As of this writing, depending on the application and the model, I've seen the percentage of correctly generated JSON objects anywhere between 0% and up to the high 90%.

## Constrained sampling

*Constraint sampling* is a technique for guiding the generation of text toward certain constraints. It is typically followed by structured output tools.

At a high level, to generate a token, the model samples among values that meet the constraints. Recall that to generate a token, your model first outputs a logit vector, each logit corresponding to one possible token. Constrained sampling filters this logit vector to keep only the tokens that meet the constraints. It then samples from these valid tokens. This process is shown in [Figure 2-21](#).



*Figure 2-21. Filter out logits that don't meet the constraints in order to sample only among valid outputs.*

In the example in [Figure 2-21](#), the constraint is straightforward to filter for. However, most cases aren't that straightforward. You need to have a grammar that specifies what is and isn't allowed at each step. For example, JSON grammar dictates that after `{`, you can't have another `{` unless it's part of a string, as in `{"key": "{{string}}"}.`

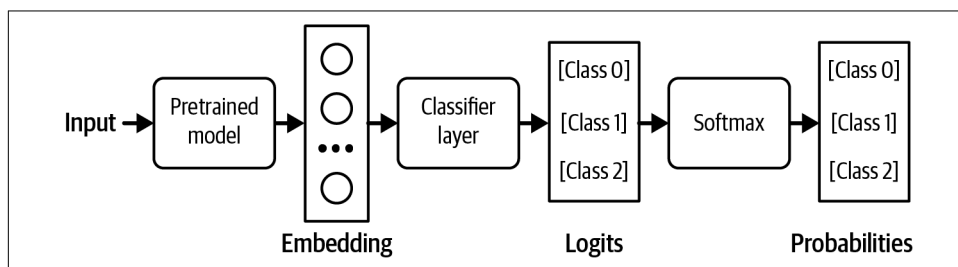
Building out that grammar and incorporating it into the sampling process is nontrivial. Because each output format—JSON, YAML, regex, CSV, and so on—needs its own grammar, constraint sampling is less generalizable. Its use is limited to the formats whose grammars are supported by external tools or by your team. Grammar verification can also increase generation latency ([Brandon T. Willard, 2024](#)).

Some are against constrained sampling because they believe the resources needed for constrained sampling are better invested in training models to become better at following instructions.

## Finetuning

Finetuning a model on examples following your desirable format is the most effective and general approach to get models to generate outputs in this format.<sup>34</sup> It can work with any expected format. While simple finetuning doesn't guarantee that the model will always output the expected format, it is much more reliable than prompting.

For certain tasks, you can guarantee the output format by modifying the model's architecture before finetuning. For example, for classification, you can append a classifier head to the foundation model's architecture to make sure that the model outputs only one of the pre-specified classes. The architecture looks like [Figure 2-22](#).<sup>35</sup> This approach is also called *feature-based transfer* and is discussed more with other transfer learning techniques in [Chapter 7](#).



*Figure 2-22. Adding a classifier head to your base model to turn it into a classifier. In this example, the classifier works with three classes.*

During finetuning, you can retrain the whole model end-to-end or part of the model, such as this classifier head. End-to-end training requires more resources, but promises better performance.

We need techniques for structured outputs because of the assumption that the model, by itself, isn't capable of generating structured outputs. However, as models become more powerful, we can expect them to get better at following instructions. I suspect that in the future, it'll be easier to get models to output exactly what we need with minimal prompting, and these techniques will become less important.

<sup>34</sup> Training a model from scratch on data following the desirable format works too, but this book isn't about developing models from scratch.

<sup>35</sup> Some finetuning services do this for you automatically. [OpenAI's finetuning services](#) used to let you add a classifier head when training, but as I write, this feature has been disabled.